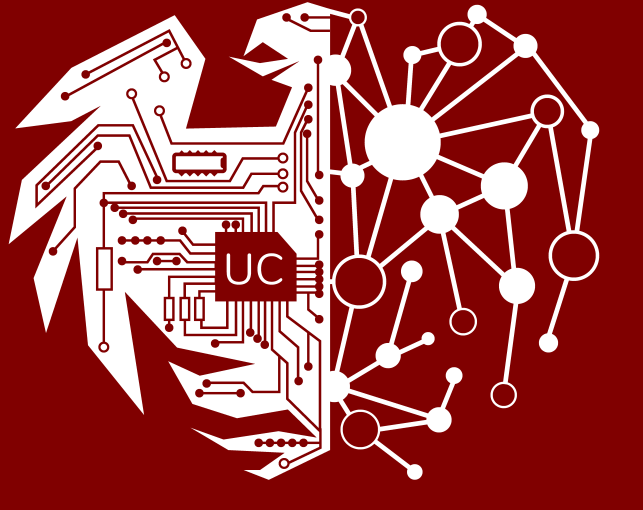




UpDown: Efficient Manycore based on Many Threading and Scalable Memory Parallelism



Andronicus Rajasukumar¹ Ruiqi Xu¹ Tianchi Zhang¹ Yuqing (Ivy) Wang¹ Tianshuo Su² Marziyeh Nourian¹ Jianru Ding¹

Jiya Su¹ Rajat Khandelwal¹ Alexander Fell¹ David F. Gleich³ Yanjing Li⁴ Henry Hoffmann¹ Andrew A. Chien^{1,5}

¹University of Chicago

²Google LLC

³Purdue University

⁴Northeastern University

⁵Argonne National Laboratory

Motivation

Traditional cores achieve high performance when they have a steady stream of useful instructions, as well as low memory latency and high data reuse. However, irregular applications break these assumptions, causing low datapath utilization and associated poor performance.

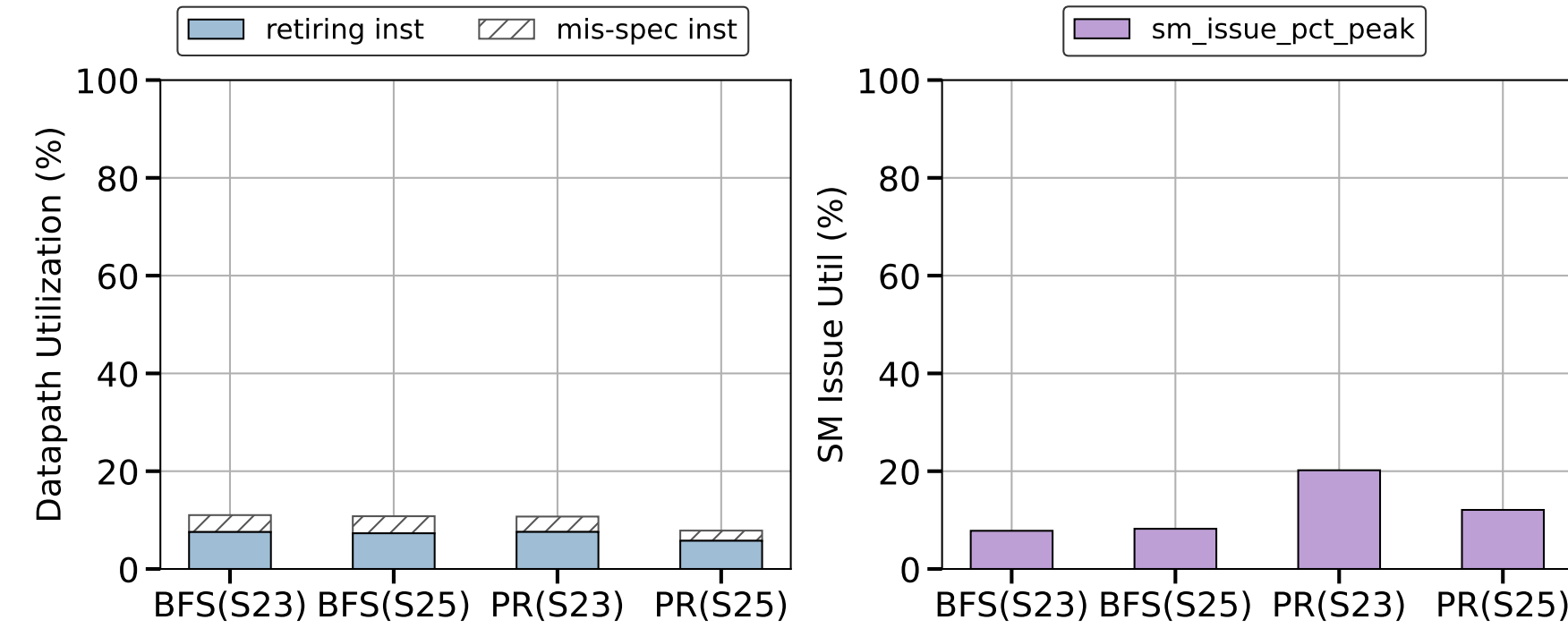


Figure 1. Irregular workloads show poor datapath utilization on both CPUs and GPUs.

- Irregular applications have many branches and unpredictable data-driven control flows, leading to ineffective branch prediction and speculation.
- Complex pointer/indirection structures and random data accesses reduce the effectiveness of prefetching, leading to high cache miss rates and pipeline stalls.
- High-bandwidth memory is only useful if the architecture can keep enough requests in flight, but the achievable memory-level parallelism is limited by physical resources like MSHR.

UpDown Approach

Fill Datapath with Useful Instructions from Short Thread Invocations

UpDown achieves high utilization by exploiting many threads and high memory-level parallelism.

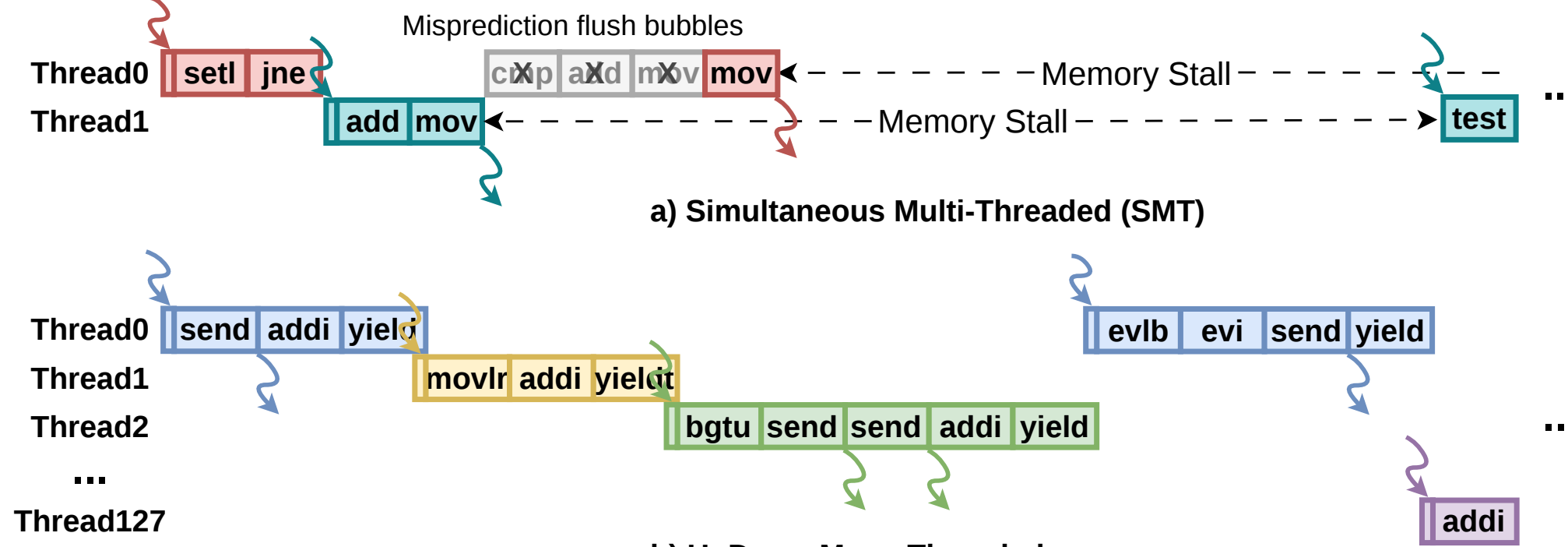


Figure 2. Many-thread execution keeps the pipeline busy despite irregular control and memory stalls.

- UpDown has 128 hardware threads available to fill the datapath, converting unpredictable branches into many-thread scheduling.
- Each thread runs 10-100 instructions at a time under explicit software control. Fast hardware support reduces switching cost to a single cycle.

Generate Scalable Memory Level Parallelism

UpDown provides instructions for non-binding, split-transaction DRAM access. Critically, UpDown's novel DRAM operations do not bind outstanding DRAM access to any hardware state element (e.g., register or MSHR), effectively creating unlimited memory parallelism.

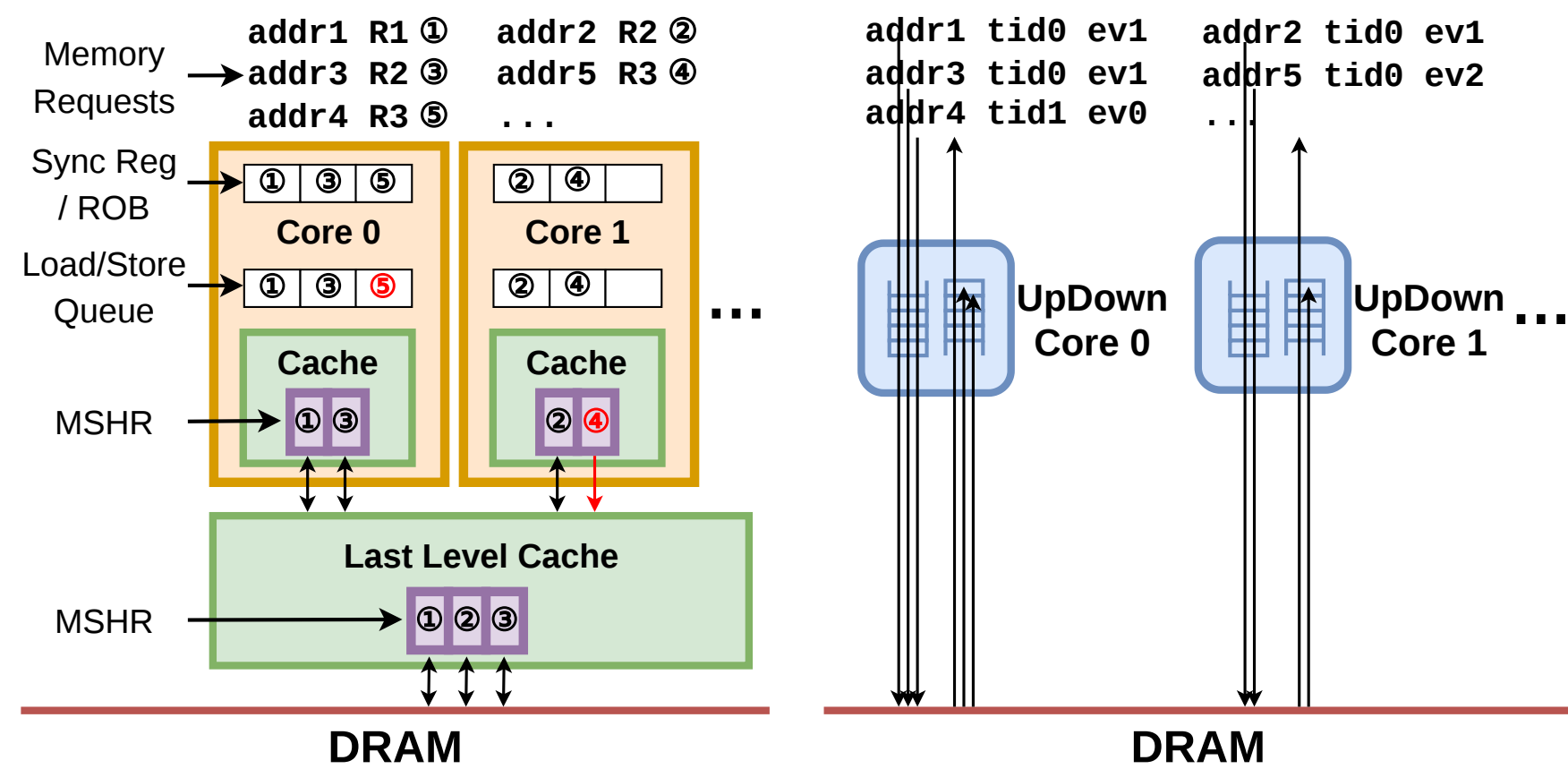


Figure 3. Conventional cores bind each miss to ROB, LSQ, and MSHR state, while UpDown uses software names and event-driven responses to avoid those limits.

- UpDown does not bind DRAM requests to resources, so unlimited memory requests can be in flight.
- Software names trigger appropriate computation and synchronization for returning data, for example to count responses, accumulate values, or specify the offset where the data should be stored.

UpDown Architecture

Key Architecture Mechanisms

- Event-Driven Scheduling (EDS):** UpDown threads are scheduled as events (short invocations), enabling fine-grained parallelism with hardware support for scheduling and queue management.
- Software-controlled Lightweight Threading (SLT):** UpDown has 128 hardware thread contexts, each with a small state. The threads are created, suspended, and terminated under software control.
- Split-transaction Memory Access with software synchronization (SMA):** Memory accesses are decoupled from thread execution (no tracking/bookkeeping) and synchronized using software names, enabling high and scalable memory level parallelism.

Instruction Set Architecture

- Events** are first-class primitives and provide a unified interface to hardware (e.g., DRAM responses) and software events. An event word determines the instruction sequence and the compute location (thread and lane ID).
- State:** Each core has 128 hardware thread contexts; 16 registers (GPRs) each. The events have up to 9 message operands integrated directly into the register namespace to avoid copying.

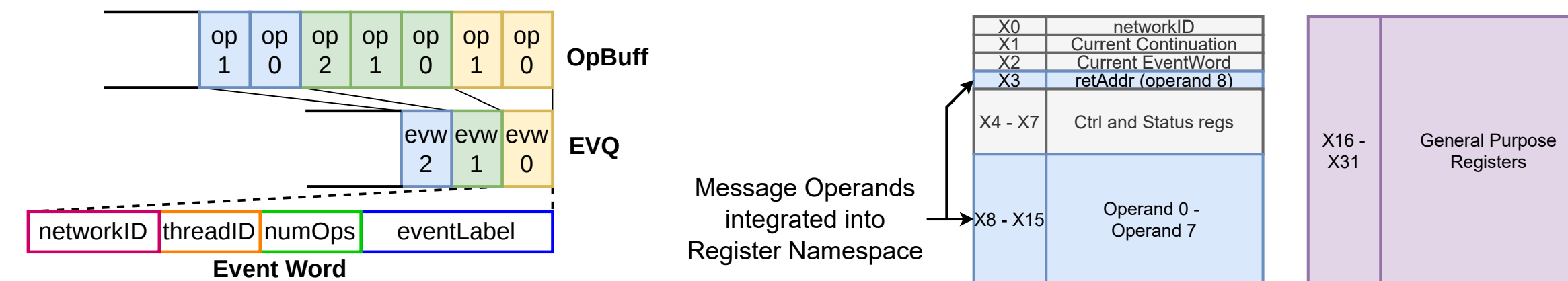


Figure 4. Incoming events carry both control and data, reducing thread startup overhead.

- Thread control instructions (`yield`, `yieldt`) provide zero-cycle thread creation and scheduling and single-cycle thread suspension and termination.
- `send`, `sendr`, and `sendops` launch events across the chip. `sendm` issues non-blocking DRAM operations.

Lane Micro-architecture

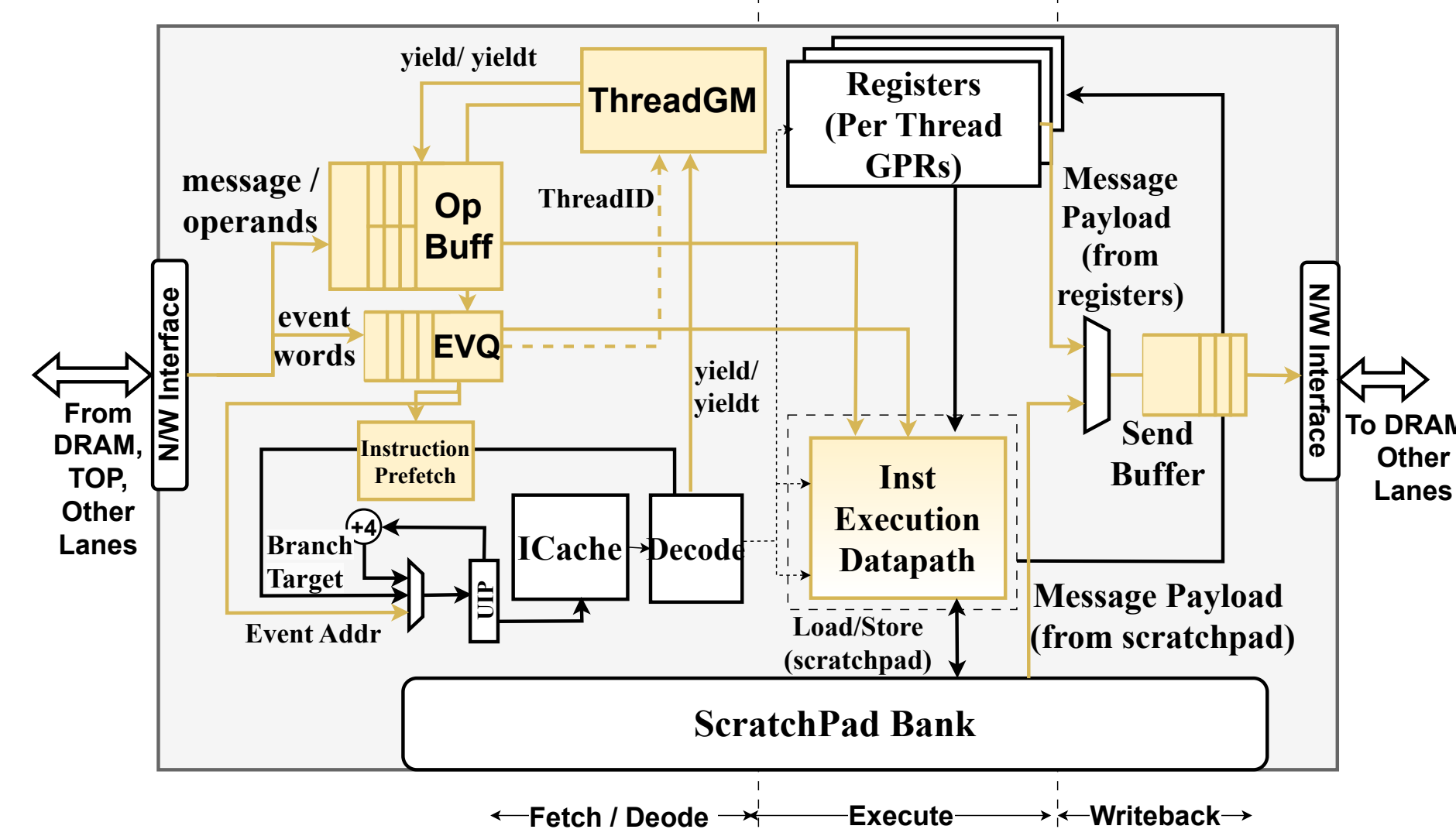


Figure 5. UpDown Micro-architecture

- Each core executes a sequence of events, taken from its local Event Queue (EVQ) in FIFO order, which is simple and energy-efficient. The operands are stored in the OpBuff and mapped into the register namespace.
- The threadID used by the ThreadGM to either activate an existing thread context or allocate a new one if it is `0xff`. FIFO scheduling enables easy identification of the next thread and its instructions, enabling **single-cycle activation**.
- Send instructions format a message using data from scratchpad, registers, or OpBuff, pushing them through the `SendBuffer` into the NoC. The event_word includes a networkID that is used for routing.
- Programs can identify variable latency operations or unpredictable branches, and release the datapath. `yield`, `yieldt` instructions suspend, terminate the current thread, respectively

Chip Level Integration

The evaluated UpDown chip integrates 2,048 cores and 4 HBM stacks through a hierarchical NoC.

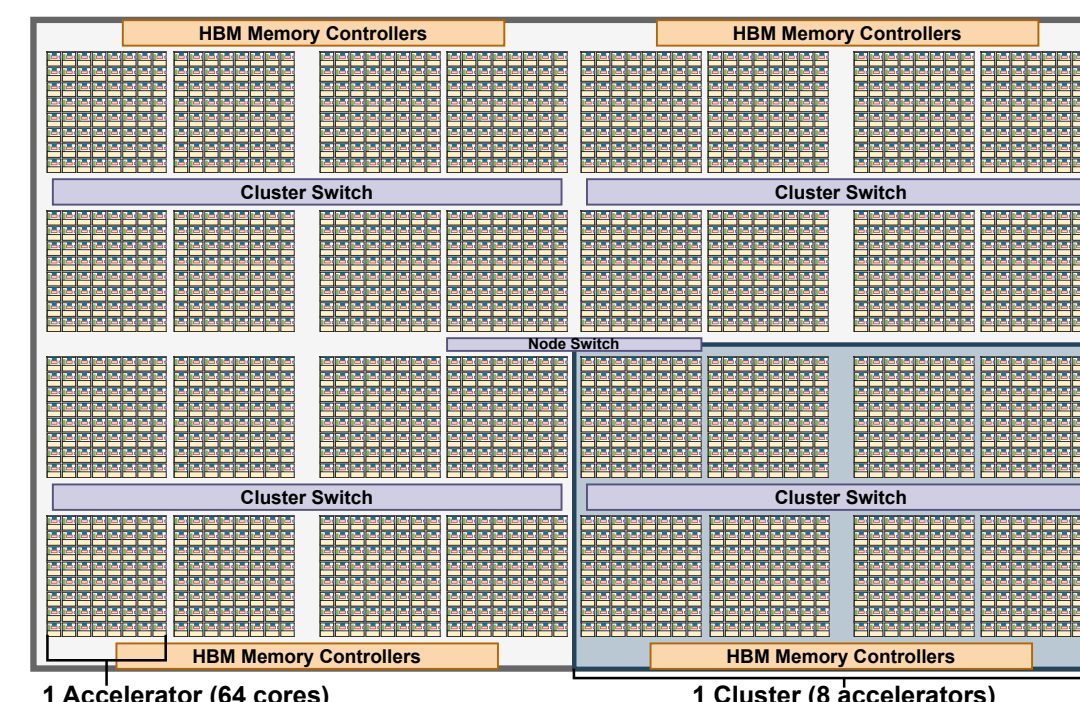


Figure 6. Chip organization: 4 clusters, 32 accelerators, 2,048 cores, and 4 HBM stacks.

Performance Comparison to a State-Of-The-Art ManyCore

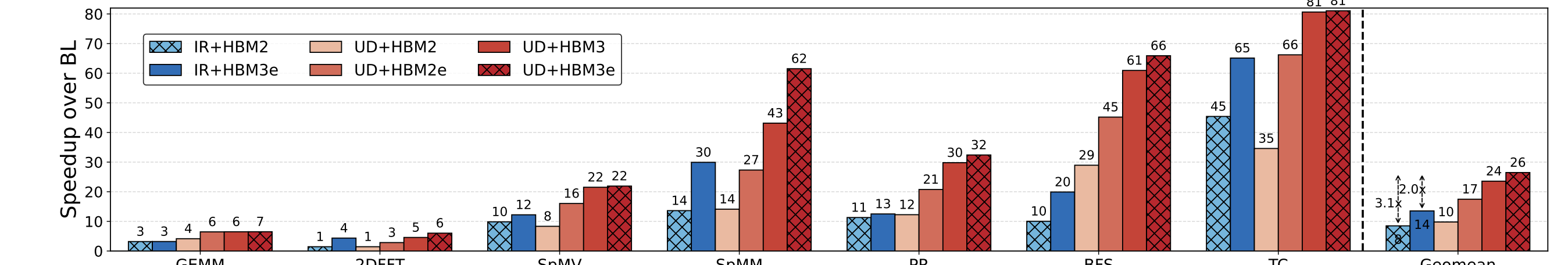


Figure 7. 2K-core UpDown versus an iso-area HammerBlade like in-order manycore (8K-cores) across HBM2 and HBM3e systems.

We compare UpDown to an 8,192-core in-order manycore (IR) modeled after HammerBlade with 2.7x issue rate.

- UpDown outperforms the iso-area in-order design on every workload by **1.2x–3.3x** with HBM3e, with **2.0x** geomean improvement over IR+HBM3e and **3.1x** over IR+HBM2
- UpDown gains **2.6x** speedup from HBM2 to HBM3e, showing it translates additional bandwidth to useful work.

Understanding the Impact of UpDown Mechanisms on Performance

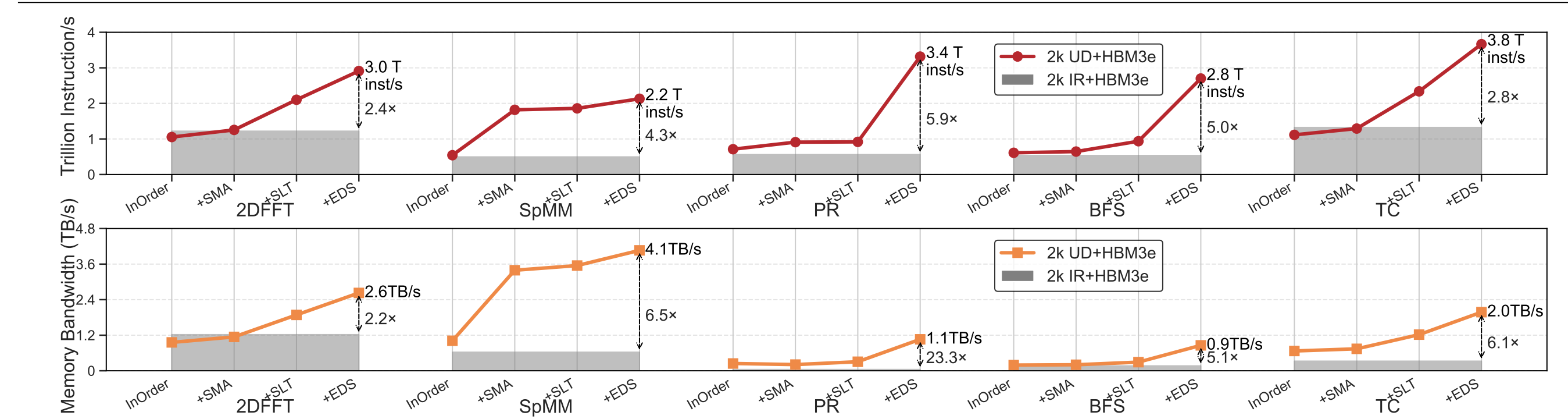


Figure 8. Normalized study showing how EDS, SLT, and SMA increase compute throughput and memory bandwidth.

- Against a normalized in-order design, UpDown's mechanisms deliver **2.4x–5.9x** performance gains depending on the workload.
- Memory utilization improves by **2.2x–23.3x**, explaining why UpDown benefits more from HBM3e than the in-order manycore.
- Averaged across workloads, EDS contributes **1.9x**, while SLT and SMA each contribute about **1.4x**.

Scaling Compute and Memory Parallelism

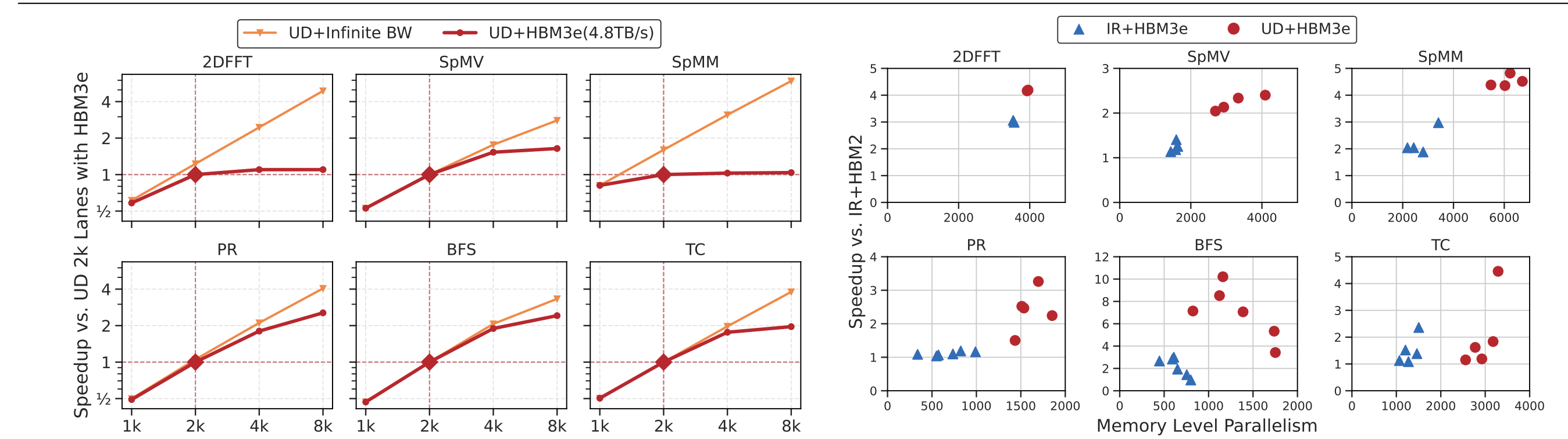


Figure 9. Scaling trends: more cores help until bandwidth saturates, and UpDown reaches much higher memory-level parallelism than the in-order manycore.

- 2K cores are already a good match for several workloads; 4K cores in a denser process could exploit even more HBM3e bandwidth.
- Despite having only one quarter as many cores as IR, UpDown reaches much higher MLP and therefore much better performance on irregular applications.

Summary

- UpDown is an efficient manycore building block, achieving high performance by keeping the datapath busy and creating enough memory-level parallelism to use HBM effectively.
- UpDown's key mechanisms allow a 2K-core UpDown to outperform an iso-area HammerBlade-style 8K-core in-order manycore.

Acknowledgements This work was supported by ODNI/IARPA through the AGILE program under ARO contract W911NF22C0082, and in part by a Computation Innovation Fellows Award.

Selected References

- A. Rajasukumar et al., "UpDown: Efficient Manycore based on Many Threading and Scalable Memory Parallelism," ICS '26.
- A. Rajasukumar et al., "UpDown: A Novel Architecture for Unlimited Memory Parallelism," MEMSYS '24.
- A. Chien et al., "UpDown Accelerator Instruction Set Architecture (ISA) v2.4," UChicago TR-2024-03.